



MASSIVE CHEAT SHEET

Configured specifically for your `init.lua` baseline & plugin workflow

Leader: `<Space>`

Clipboard: `unnamedplus`

Relativenumber: `On`

Colorscheme: `vscode_black`

LSP: TS, Py, Java21, C++, HTML, CSS, JSON, ESLint

Cmp: `blink.cmp`

Explorer: `oil.nvim`

☀ YOUR EXPLICIT CUSTOM KEYMAPS (DEFINED IN INIT.LUA)

SPACE LEADER

<code>Space</code> <code>f</code> <code>f</code>	Find Files: Open Telescope fuzzy finder to search files by filename
<code>Space</code> <code>f</code> <code>g</code>	Live Grep: Search text dynamically across entire project with Telescope
<code>-</code>	Oil File Explorer: Open parent directory as editable buffer (stevearc/oil.nvim)
<code>Space</code> <code>f</code>	Format Buffer: Format active buffer asynchronously using connected LSP server
<code>Space</code> <code>r</code> OR <code>F5</code>	Run Java File: Auto-saves & compiles/runs Java in 14-line terminal split (JDK 21)
<code>Space</code> <code>d</code>	Diagnostic Details: Show floating error/warning detail window under cursor
<code>Space</code> <code>q</code>	List Diagnostics: Populate & open location list with all project diagnostics
<code>]</code> <code>d</code>	Next Diagnostic: Jump to next error/warning diagnostic in buffer
<code>[</code> <code>d</code>	Prev Diagnostic: Jump to previous error/warning diagnostic in buffer
<code>Space</code> <code>t</code> <code>w</code>	Toggle Warning Display: Show/hide warning virtual text & underlines on demand
<code>Esc</code>	Clear Search Highlighting: Remove highlight search selection (nohlsearch)

🔍 FUZZY SEARCH & FILE EXPLORER

<code>Space</code> <code>f</code> <code>f</code>	Telescope: Search files in project
<code>Space</code> <code>f</code> <code>g</code>	Telescope: Live grep text in project
<code>Ctrl</code> + <code>n</code> / <code>Ctrl</code> + <code>p</code>	Telescope: Select next / previous result
<code>Enter</code>	Telescope: Open file in current window
<code>Ctrl</code> + <code>x</code> / <code>Ctrl</code> + <code>v</code>	Telescope: Open in Horizontal / Vertical split
<code>Ctrl</code> + <code>t</code>	Telescope: Open file in new tab page
<code>-</code>	Oil.nvim: Open file explorer in buffer
<code>Enter</code> (in Oil)	Oil.nvim: Open file / enter directory
<code>_</code> (in Oil)	Oil.nvim: Open current working directory
<code>g</code> <code>.</code> (in Oil)	Oil.nvim: Toggle hidden files & dotfiles

⚡ BLINK.CMP & GITSIGNS

<code>Ctrl</code> + <code>Space</code>	Blink: Manually open completion popup
<code>Tab</code> / <code>Shift</code> + <code>Tab</code>	Blink: Next / Prev item or snippet jump
<code>Enter</code>	Blink: Accept & confirm selected completion
<code>Ctrl</code> + <code>e</code>	Blink: Cancel & hide completion menu
<code>Ctrl</code> + <code>k</code> / <code>Ctrl</code> + <code>j</code>	Blink: Scroll documentation popup up/down
<code>]</code> <code>h</code> / <code>[</code> <code>h</code>	Gitsigns: Jump to Next / Prev git hunk
<code>Space</code> <code>h</code> <code>s</code>	Gitsigns: Stage hunk under cursor
<code>Space</code> <code>h</code> <code>r</code>	Gitsigns: Reset/revert hunk under cursor
<code>Space</code> <code>h</code> <code>p</code>	Gitsigns: Preview hunk inline diff float
<code>Space</code> <code>h</code> <code>b</code>	Gitsigns: Show git blame for current line

🧠 LSP CODE INTELLIGENCE

K	Hover: Show type definition & documentation float
g d	Go to Definition: Jump to symbol definition
g D	Go to Declaration: Jump to symbol declaration
g r i Or g i	Go to Implementation: Find interface implementation
g r r Or g r	Go to References: Find all references of symbol
g r n	Rename Symbol: Smart refactoring across workspace
g r a	Code Action: Open LSP quick fixes & quick actions
Ctrl + k (Insert)	Signature Help: Display function arguments tooltip
Space f	Format: LSP async formatting for active file

👁 MOTION & CURSOR NAVIGATION

h / j / k / l	Basic motions: Left / Down / Up / Right
w / b	Jump forward / backward by word start
e / g e	Jump forward / backward to word end
0 / ^ / \$	Start of line / First non-blank / End of line
g g / G	Jump to Top of file / Bottom of file
{ N } G	Jump directly to line number N
{ / }	Jump to previous / next paragraph block
%	Jump to matching bracket (, [, {}, {}
f {char} / t {char}	Find character in line / Till character in line
; / ,	Repeat last character find / Repeat in reverse
Ctrl + u / Ctrl + d	Smooth scroll half page Up / Down
z z / z t / z b	Recenter screen / Screen to Top / Screen to Bottom

✍ EDITING, DELETING & CHANGING

i / a	Insert before cursor / Append after cursor
I / A	Insert at start of line / Append at end of line
o / O	Open new line below / Open new line above
x / X	Delete character under cursor / Delete character before
d d / D	Delete entire line / Delete to end of line
d w / d b	Delete word forward / Delete word backward
c c / C	Change entire line / Change to end of line
c w	Change word (deletes word & enters insert mode)
r {char} / R	Replace single character / Enter overwrite replace mode
u / Ctrl + r	Undo last change / Redo last undone change
.	Repeat last editing command

📦 POWERFUL TEXT OBJECTS

c i w / d a w	Change inner word / Delete around word (with space)
c i " / c a "	Change inside quotes / Change around quotes
c i ' / c i `	Change inside single quotes / inside backticks
c i (/ d a (	Change inside parens / Delete around parens (
c i { / d a {	Change inside braces / Delete around braces { }
c i [/ d a [	Change inside brackets / Delete around brackets []
c i t / d a t	Change inside HTML tag / Delete around HTML tag
d i p / d a p	Delete inner paragraph block / Delete around paragraph

✦ VISUAL SELECTION & CLIPBOARD

<code>v</code> / <code>V</code>	Visual character selection / Visual line selection
<code>Ctrl</code> + <code>v</code>	Visual Block selection (Vertical multi-line editing)
<code>y</code> / <code>p</code>	Yank selection / Paste (Auto-syncs with Windows Clipboard!)
<code>P</code>	Paste before cursor position
<code>"</code> <code>0</code> <code>p</code>	Paste last yanked text (Bypasses deleted text register)
<code>></code> / <code><</code>	Indent selected lines right / Unindent left
<code>=</code>	Auto-indent selected code block according to rules
<code>u</code> / <code>U</code>	Convert selected block to lowercase / uppercase
<code>g</code> <code>Ctrl</code> + <code>a</code>	Create sequential incrementing numbers in block mode

📑 WINDOW SPLITS, BUFFERS & TABS

<code>Ctrl</code> + <code>w</code> <code>s</code>	Horizontal Split: Open split window below
<code>Ctrl</code> + <code>w</code> <code>v</code>	Vertical Split: Open split window to right
<code>Ctrl</code> + <code>w</code> <code>h/j/k/l</code>	Window Focus: Move focus Left / Down / Up / Right
<code>Ctrl</code> + <code>w</code> <code>q</code> OR <code>:q</code>	Close Split: Close current window split
<code>Ctrl</code> + <code>w</code> <code>o</code>	Close Others: Close all window splits except active
<code>Ctrl</code> + <code>w</code> <code>=</code>	Equalize Window Sizes: Balance all splits equally
<code>:bnext</code> / <code>:bprev</code>	Buffer Nav: Switch to Next / Previous open buffer
<code>:bdelete</code> or <code>:bd</code>	Delete Buffer: Close active file buffer
<code>g</code> <code>t</code> / <code>g</code> <code>T</code>	Tab Nav: Switch to Next tab / Previous tab

🔍 SEARCH, MARKS & MACROS

<code>/</code> <code>pattern</code>	Search forward for pattern (case-smart)
<code>n</code> / <code>N</code>	Jump to next search match / previous match
<code>*</code> / <code>#</code>	Search word under cursor forward / backward
<code>Esc</code>	Clear search highlighting (Custom mapped)
<code>:%s/old/new/g</code>	Replace all occurrences of 'old' with 'new' in file
<code>:%s/old/new/gc</code>	Replace with interactive confirm prompt (y/n/a/q)
<code>m</code> <code>{a-z}</code> / <code>'</code> <code>{a-z}</code>	Set mark {a-z} / Jump to mark line
<code>q</code> <code>{a-z}</code> / <code>q</code>	Start recording macro into register {a-z} / Stop
<code>@</code> <code>{a-z}</code> / <code>@</code> <code>@</code>	Play recorded macro {a-z} / Repeat last macro

🚀 JAVA RUNNER & TERMINAL WORKFLOW

<code>Space</code> <code>r</code> OR <code>F5</code>	Run Java: Saves file, splits bottom terminal (14 lines), executes with JDK 21
<code>Ctrl</code> + <code>\</code> <code>Ctrl</code> + <code>n</code>	Exit Terminal Mode: Return from terminal input to Normal mode
<code>i</code> OR <code>a</code> (in terminal)	Enter Terminal Mode: Re-enable terminal input (e.g. for Scanner prompt)
<code>:terminal</code>	Open integrated terminal in current window
<code>TextYankPost</code>	Visual feedback flash (150ms) whenever text is copied
<code>undofile = true</code>	Persistent undo history across Neovim restarts